delay()

From the very beginning with our first Blink sketch, we have made use of the `delay()` function to create a short pause in the middle of a program. There's really not a whole lot to the function, but there are some things we need to be aware of. The syntax for the function follows.

```
delay(time)
```

Time is specified in milliseconds, where a delay of 1000 milliseconds equals 1 second, 2000 milliseconds equals 2 seconds, and so on. This value can be expressed as a constant or variable in the unsigned long data type. Yes, that means as an unsigned long, it is theoretically possible to express a delay of 4,294,967,295 milliseconds, or roughly seven weeks long. In the interest of full disclosure, I have not personally verified this.

The delay time period needs to be expressed in a positive or unsigned value, or the `delay()` function will freak out and do things you would not expect, like rolling over a negative number to a really large positive number—and then you find yourself waiting seven weeks to find out. Likewise, you might, possibly through an arithmetic operation, specify a time delay that is equal to the expression 60 * 1000 in an attempt to create a delay that is 1 minute long. Because both of these values are expressed as signed integer constants, the result of 60 * 1000 is not 60,000 as you might expect, but rather something like -5, 536. This is a little glitch in how these values are processed—because both are signed integers, the result is also expressed as a signed integer.

Since now is as good of a time as any, let's look at how to fix this problem by forcing a specific data type using an integer formatter to get a result more compatible with our function. Table 7-1 provides several integer formatters that pertain to this discussion.

Table 7-1. *Integer Constant Data Type Formatters*

Formatter	Example	Description
"u" or "U"	255u	Forces constant into an unsigned data type
"l" or "L"	1000L	Forces constant into a long data type
"ul" or "UL"	32767ul	Forces constant into an unsigned long-data type

Given our example of wanting to create a hypothetical delay of 60 seconds, or 1 minute, through the expression 60 * 1000, we would need to force one of these constants into a long data type to keep the value in the positive range by tacking on the `L` formatter, as in: `60 * 1000L`. Because one of the values in this expression is now of the larger, long-data type, results from this expression will remain in the long data type, giving us the value of 60,000 that we were looking for.

Now that we are somewhat aware of problems with excessive delays created by the wrong data types, we can move on to using counter variables in place of integer constants to create a changing delay period. Take the following code sample:

```
for (int i=250; i>0; i-=5) {
  digitalWrite(13, HIGH);
  delay(i);
  digitalWrite(13, LOW);
  delay(i);
}
```